

ISISI — team cheat sheet

Altur HackMTY 2026 · synthetic-caller detection · everything on one page

The essentials

CHALLENGE	“Defend the Bank Against Voice Deepfakes.” Given a stereo 8 kHz call (ch 0 = caller, ch 1 = Altur agent), decide whether the caller is human or synthetic. <code>POST /detect</code> → <code>{"is_synthetic": bool, "confidence": float}</code> . Judged on a hidden test set, on robustness, feasibility and latency.
DATASET	353 calls — train 282 (113 human / 169 synthetic), val 71 (37 / 34). 8 kHz · stereo · PCM_16 · WAV, all 353, no exceptions. 14.5 h audio, of which only 3.9 h is caller speech . No speaker ids, no TTS ids, no codec fields.
FINAL MODEL	Robust Acoustic V2 — <code>models/robust_v2/</code> . Frozen Wav2Vec2 Spanish, hidden layer 5, Silero VAD, trained on original + 29 h of simulated telephone-channel audio. <code>reports/deployed_model.json</code> is the single pointer both <code>predict.py</code> and <code>server.py</code> read.
PREPROCESSING	channel 0 → VAD → speech regions → 4 s chunks (drop < 1 s, cap 60, never cross a region) → RMS-normalise to -26 dBFS → resample 8 k → 16 kHz (polyphase, exact 2×).
BACKBONE	<code>facebook/wav2vec2-base-es-voxpopsli-v2</code> — 94.37 M params, 12 layers × 768, pretrained self-supervised on 21 400 h of Spanish. Frozen , and physically truncated after layer 5 (blocks 6–12 deleted at load: half the compute gone, identical output).
SELECTED LAYER	5 of 12 (42 % depth) . Chosen on stressed AUC — the clean split was useless for this, because <i>every</i> layer of <i>every</i> backbone scored AUC 1.000, including layer 0.
CLASSIFIER	standardise (buffers in the checkpoint) → <code>Linear(768→256)</code> → ReLU → Dropout 0.3 → <code>Linear(256→2)</code> . Score = <code>logit[1] - logit[0]</code> . 197 378 trainable params vs 94 371 712 frozen = 0.21 % .
AGGREGATION	arithmetic mean of the chunk log-odds , fixed in config before results were seen. Then Platt: <code>σ(0.9025 · s + 0.8366)</code> .
TRAINING	seed 42 · AdamW · lr 1e-3 · weight decay 1e-4 · batch 64 · class-weighted cross-entropy · max 60 epochs, early stop patience 15 on val loss → stopped at 18, kept epoch 3 . ~10 s on cached embeddings, RTX 4080 Laptop.
CALIBRATION	Platt, fitted on 781 call-conditions (71 calls × 11 stress conditions), grouped 5-fold. Brier 0.0200 → 0.0196, ECE 0.0190 → 0.0150. Fitting on the clean split was degenerate and was thrown away.
LATENCY	119 ms GPU inference (850 ms CPU). Confident verdict after a median 2.33 s of caller speech , 12.3 s into the call, 132.7 s of lead time . 100 % of calls confident before halfway; early verdict matched final 100 % of the time.
API	<code>POST /detect</code> (8 accepted field names, multipart, or raw base64; always HTTP 200). Plus <code>/detect_layers</code> , <code>/fusion/config</code> , <code>/fusion/recombine</code> , <code>/layers</code> , <code>/demo/*</code> , <code>/twilio/*</code> , <code>/api/{calls,stats,health}</code> , and three pages. One backend: <code>python src/server.py</code> .

The numbers to quote

HELD-OUT SPLIT (71 CALLS)	ACCURACY	AUC	BRIER
Acoustic V1 alone	100 %	1.000	0.0003
Behaviour alone	97.2 %	0.991	0.0406

HELD-OUT SPLIT (71 CALLS)	ACCURACY	AUC	BRIER
Semantic alone	91.5 %	0.975	0.0698
Fused (50/50 + gated verifier)	100 %	1.000	0.0096

ROBUSTNESS, ACCURACY AT THE DEPLOYED THRESHOLD	SPECIALIST V1	ROBUST V2
Altur validation, clean	100.0 %	100.0 %
Through a line, SEEN codecs	77.5 %	100.0 %
Through a line, UNSEEN codecs (GSM, AMR, iLBC, Speex, Opus, G.722)	81.7 %	100.0 %
11 channel perturbations	97.4 %	98.5 %
Off-pipeline TTS down a line	62.0 %	81.5 %
Off-pipeline TTS, clean (<i>the one V2 loses</i>)	68.5 %	54.6 %

The honest number: 88.0 % accuracy / AUC 0.945 on 108 out-of-dataset clips band-passed through a simulated line. Quote this, not the 100 %.

The three things that will be asked

BIGGEST SHORTCUT DISCOVERED

A logistic regression on **24 call statistics with no voice content at all** reaches validation **AUC 1.000, accuracy 98.6 %**. The strongest single feature is **leading silence before the caller's first word**: median 1.16 s for humans, **10.38 s** for synthetic, AUC 0.948. Then the high-band energy ratio, loudness (~6 dB), and response latency. **You can saturate this split without hearing a phoneme**, so a perfect score on it is not evidence of voice modelling. RMS normalisation puts loudness and timing out of the acoustic model's reach; the band edge it cannot escape, which is what a codec then destroys.

BIGGEST LIMITATION

71 calls cannot establish banking error rates — the exact one-sided 95 % lower bound on 71/71 is **95.9 %** (94.9 % two-sided) — and **no audio in this project has been through a real carrier** — every telephone result comes from our own simulator. Also: not adversarially tested, not streaming, and the semantic layer sends audio to a third party.

WHY OUR SOLUTION IS DIFFERENT

Most teams would have stopped at 100 %. We treated it as a defect: eight sanity checks, a label permutation test that came back at chance, a trivial-statistics model that matched our neural one, a telephone channel built specifically to break our own detector, and an entire codec family held out of training to prove the fix generalised. Then we shipped the model that is **worse on one benchmark** because it is better where a bank actually operates — and we printed both criteria rather than the flattering one.

The 30-second version

*A bank's phone line needs to know if it is talking to a machine. We built three independent detectors — the voice, the conversation timing, and the words — and fused them. The voice model hit 100 % on validation immediately, which we treated as a warning rather than a win: we proved you can score 100 % on this dataset using call statistics alone, without hearing a voice. So we built a telephone-channel simulator, broke our own model with it (100 % down to 70 %), and retrained through 29 hours of simulated phone lines. The deployed model now holds 100 % through codec families it was never trained on, and commits to a verdict after about **two and a half seconds of caller speech**, more than two minutes before the call ends.*

The 2-minute version

The problem. Voice cloning is cheap now, and a contact centre is a bank's front door. We get a stereo 8 kHz call: channel 0 is the caller we must classify, channel 1 is the Altur agent.

The first model. Frozen Wav2Vec2 Spanish, hidden layer 5, mean-pooled over 4-second chunks of caller speech, a 197 000-parameter MLP on top, Platt-calibrated. It scored 100 % on the validation split. So did every other backbone we tried, and every layer, including the raw convolutional front end.

Why we did not celebrate. A benchmark that gives every candidate the same answer cannot rank candidates. We ran eight sanity checks. Label permutation came back at chance, so it was not leakage. Ten training calls already reached 98.5 %, so it was not a hard task. Then a logistic regression on 24 call statistics — silence lengths, loudness, bandwidth, turn timing, **no voice content whatsoever** — reached AUC 1.000. That located the problem precisely: the two classes in this dataset differ by more than their voices.

The thing that made it concrete. The caller channel is digitally silent between turns — exactly -72.247 dB, the 16-bit floor, in three quarters of the calls. A real telephone line never delivers digital zero. We built a channel simulator with bit-exact G.711 verified against the reference implementation over all 65 536 sample values, eleven more codecs through ffmpeg, bursty packet loss, clock drift, comfort noise and AGC. Ten calls through it took our model from 100 % to 70 % — and the failure was asymmetric: humans held, **synthetic callers started looking human**, which is the direction that costs a bank money.

The fix. 29 hours of channel-transformed audio, with the channel drawn without ever consulting the label, and eight codec families — the parametric ones, GSM, AMR, Opus — deliberately held out of training so we could measure whether robustness generalised or just memorised. Robust V2 holds **100 % on codec families it has never heard**. It is worse than the old model on clean studio audio, because it has stopped keying on loudness and band edge, and we selected it anyway: a bank's phone line cannot produce clean studio audio.

The product. Three layers behind one endpoint. Acoustic and behaviour vote 50/50 and settle 66 of 71 calls on their own; a paid semantic verifier is consulted only for the 5 that are genuinely uncertain — on one of them the behaviour layer was confidently wrong at 0.96 and the verifier carried the verdict. One process, 219 tests, no audio stored, and a live “try being the caller” button that phones you and scores your own voice down a real line.

Numbers you must not get wrong

DO NOT SAY	SAY
"100 % accurate"	"71 of 71 on the held-out split; 95.9 % lower bound; 88.0 % off-pipeline , which is the number we stand behind"
"we detect deepfakes"	"we produce a synthetic-voice risk signal ; it is not identity verification"
"real-time streaming"	"prefix evaluation of a non-streaming model — <code>/api/health</code> returns <code>streaming: false</code> "
"we tested on real phone calls"	"we tested through a simulator ; the live Twilio demo is the only real line, and it was not used for training"
"we tuned the fusion weights"	"they are stated design choices exposed as knobs; no weighting can be justified from a split where one layer is already perfect"
"the model learned voice artefacts"	"it learned <i>something</i> that survives a codec; we never ran the band-edge ablation that would prove what"

Run it

```
python src/server.py --port 8000          # THE backend: all 3 layers, one process
cd web && npm run dev                     # landing + /admin/ + /demo/
```

Use `127.0.0.1`, never `localhost` — uvicorn binds IPv4 only. Start the server from `.venv\Scripts\python.exe`: the bare system Python has an incompatible torch, and the acoustic layer then abstains on every call while still returning HTTP 200.